

1 WHAT IS LOW LEVEL DESIGN ?

- Low Level Design (LLD) is the detailed design phase where we break down the high level design into smaller modules, classes, methods and logic.
- It focuses on how each component will be implemented.
- It is implementation-oriented.

★ Goal of LLD

- ✓ Design classes and their relationships
- ✓ Define methods and their logic
- ✓ Ensure reusability and maintainability
- ✓ Map requirements to code structure

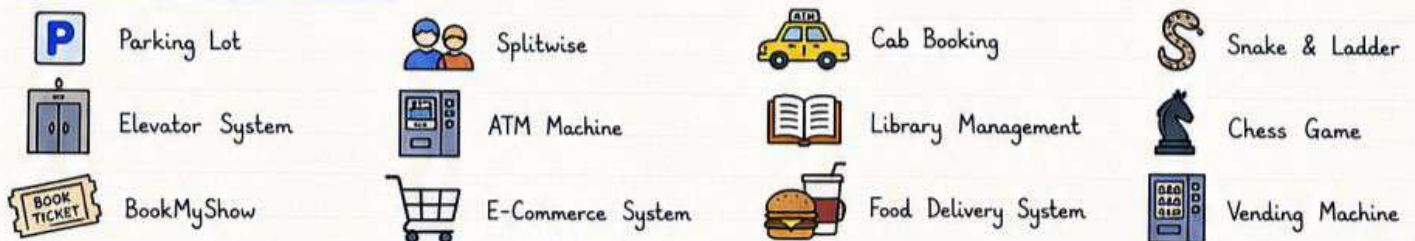
2 HLD vs LLD

Aspect	High Level Design (HLD)	Low Level Design (LLD)
Focus	Overall system architecture	Module / component level design
Level	System level	Component level
Purpose	What the system should do	How each part will do it
Output	Architecture diagrams, components	Class diagrams, method details, sequence diagrams
Abstraction	High abstraction	Low abstraction
Audience	Architects, Tech Leads	Developers
Example	Online Food Delivery System (HLD)	Restaurant, Cart, Order, Payment classes (LLD)

★ Why Companies Ask LLD ?

- To check your OOPs understanding
- To evaluate problem solving skills
- To see how you break complex problems into smaller parts
- To assess code quality and design thinking
- To ensure you can write scalable and maintainable code

3 REAL-WORLD EXAMPLES



4 LLD DESIGN PROCESS

- 1 Understand the Requirements → Ask questions, clarify doubts, understand the scope.
- 2 Identify the Entities / Classes → Identify nouns in the requirements.
- 3 Identify Relationships → Determine how classes interact with each other.
- 4 Define Attributes → List important properties for each class.
- 5 Define Methods / Operations → List the behaviors / actions for each class.
- 6 Design Class Diagram → Create UML class diagram.
- 7 Design Sequence Diagram → Show the flow of interactions.
- 8 Handle Edge Cases → Think about invalid inputs, exceptions, limits.
- 9 Review & Refine → Check for reusability, maintainability and scalability.



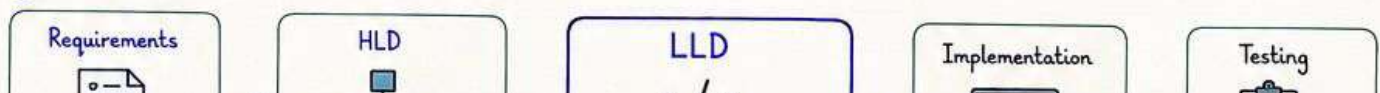
Key Tip

Always focus on:

- ✓ Good Object Design
- ✓ High Cohesion
- ✓ Low Coupling
- ✓ Code Reusability
- ✓ Scalability
- ✓ Maintainability



5 HLD → LLD : COMPLETE FLOW



SOLID principles help us write maintainable, scalable and flexible code.

Page 2 of 10

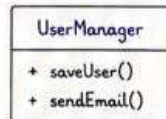
① SINGLE RESPONSIBILITY PRINCIPLE (SRP)

A class should have only one reason to change, meaning it should have only one job or responsibility.

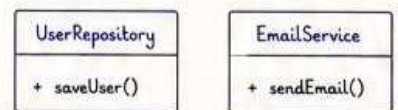
Why? Makes the class easier to understand, maintain and test.

Example:

✗ Bad (Multiple Responsibilities)



✓ Good (Single Responsibility)



Each class has one job.

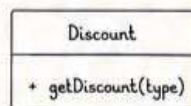
② OPEN CLOSED PRINCIPLE (OCP)

Software entities (classes, modules, functions) should be open for extension but closed for modification.

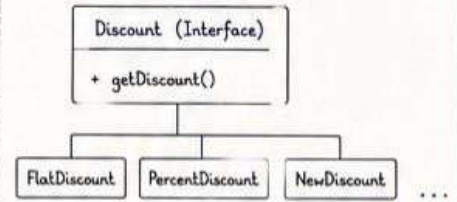
Why? We can add new functionality without changing existing code.

Example:

✗ Bad (Modification Required)



✓ Good (Extension)



Extend new classes, don't modify existing.

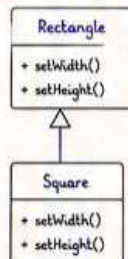
③ LISKOV SUBSTITUTION PRINCIPLE (LSP)

Objects of a superclass should be replaceable with objects of its subclasses without affecting the correctness of the program.

Why? Ensures correct behavior in inheritance hierarchies.

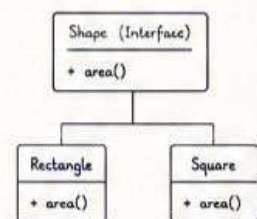
Example:

✗ Bad (LSP Violated)



Changing width may break the expected behavior of Rectangle.

✓ Good (LSP Followed)



Subclasses are safely substitutable.

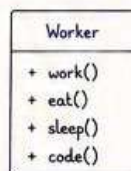
④ INTERFACE SEGREGATION PRINCIPLE (ISP)

Clients should not be forced to depend on interfaces they do not use.

Why? Better separation, smaller interfaces and easy to implement.

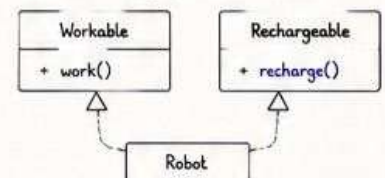
Example:

✗ Bad (Fat Interface)



Robot class forced to implement unused methods.

✓ Good (Segregated Interfaces)



Clients depend only on what they use.

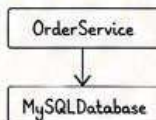
⑤ DEPENDENCY INVERSION PRINCIPLE (DIP)

High level modules should not depend on low level modules. Both should depend on abstractions.

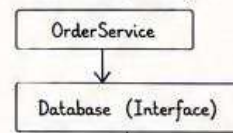
Why? Loosely coupled code that is easy to test,

Example:

✗ Bad (Direct Dependency)



✓ Good (Dependency Inversion)



Depend on abstraction, not on concrete

OOP CONCEPTS IN LLD

— The Building Blocks of Low Level Design —

"Objects are the nouns, classes are the blueprints."

Page 3 of 10

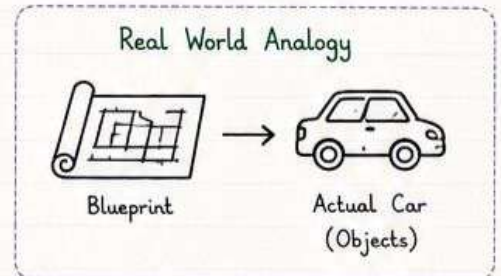
1 CLASS

- A class is a blueprint or template that defines the properties (attributes) and behaviors (methods) of objects.

Example :

Class Car is a blueprint for creating car objects.

Car (Class)
- brand : String - color : String - speed : int
+ start() : void + stop() : void + accelerate() : void



2 OBJECT

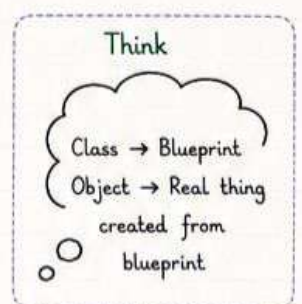
- An object is an instance of a class. It has state (values) and behavior (functionality).

Example :

obj1 and obj2 are objects of class Car.

obj1 : Car
brand = "Toyota" color = "White" speed = 0
start() stop() accelerate()

obj2 : Car
brand = "Honda" color = "Black" speed = 0
start() stop() accelerate()

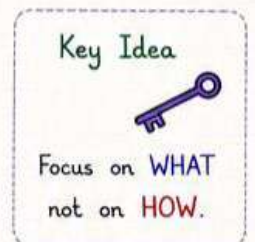
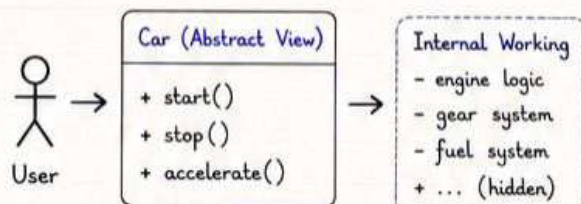


3 ABSTRACTION

- Hiding unnecessary details and showing only the essential features.

Example :

We only show what a Car does, not how the engine works.



4 ENCAPSULATION

- Bundling data (attributes) and methods that operate on the data, and restricting direct access to some of the object's components.

Example :

_speed is private, accessible only through methods.

Car
- _speed : int (private) - _engineOn : boolean (private)
+ start() : void + stop() : void + getSpeed() : int (public)

Data is hidden and can be accessed only through public methods.

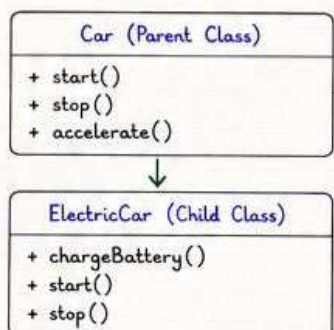


5 INHERITANCE

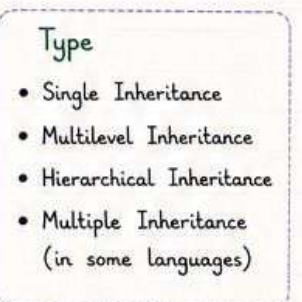
- A class can inherit properties and behaviors from another class.

Example :

ElectricCar inherits from Car class.

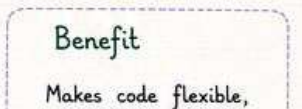
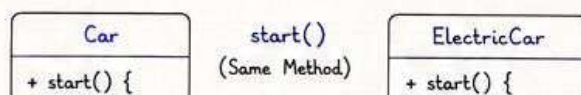


Child class reuses code from parent class.



6 POLYMORPHISM

- The ability of an object to take many forms. Same method name,

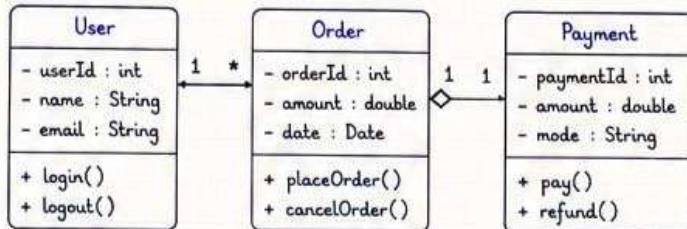


- UML (Unified Modeling Language) helps us visualize, specify, construct and document the artifacts of a software system.

1 CLASS DIAGRAM

Shows the static structure of the system. It includes classes, attributes, methods and relationships.

Example :



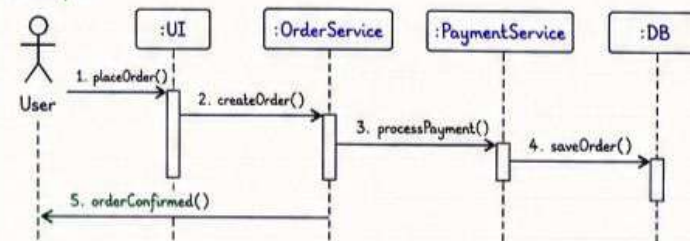
When to use?

- ✓ To model the static view.
- ✓ To understand system structure.
- ✓ To identify classes and relationships.

2 SEQUENCE DIAGRAM

Shows the interaction between objects in a particular sequence over time.

Example :



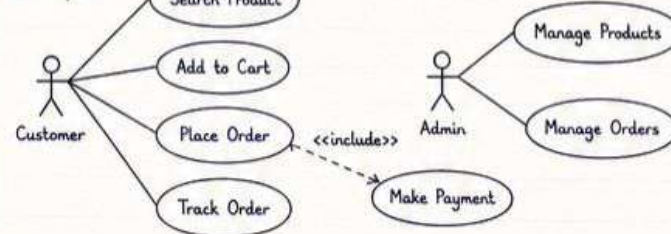
When to use?

- ✓ To show interaction between objects.
- ✓ To understand the flow of messages.
- ✓ To design use cases.

3 USE CASE DIAGRAM

Shows the high level functionality of the system from the user's perspective.

Example :



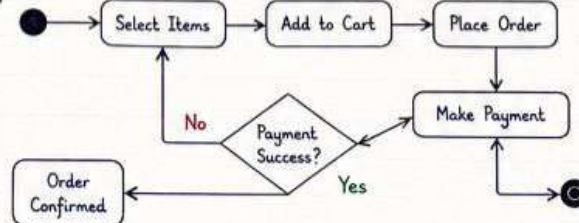
When to use?

- ✓ To capture system requirements.
- ✓ To identify actors and their goals.
- ✓ To define system boundaries.

4 ACTIVITY DIAGRAM

Shows the workflow or business process. Similar to a flowchart.

Example :



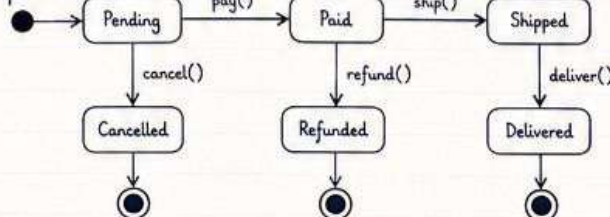
When to use?

- ✓ To model business workflows.
- ✓ To show decision points and parallel activities.
- ✓ To understand process logic.

5 STATE DIAGRAM

Shows the different states of an object and the transitions between states.

Example :



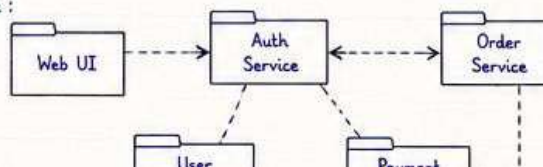
When to use?

- ✓ To model objects with different states.
- ✓ To handle event driven behaviors.
- ✓ To manage lifecycle of an object.

6 COMPONENT DIAGRAM

Shows the physical components of the system and their dependencies.

Example :



When to use?

- ✓ To show system components.
- ✓ To manage dependencies between modules.

DESIGN PATTERNS

— REUSABLE SOLUTIONS TO COMMON PROBLEMS —

"Good engineers know how to build. Great engineers know what to reuse,"

Page 5 of 10

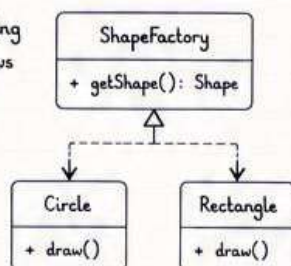
- Design patterns are proven solutions to common design problems.
- They make our code more reusable, flexible and maintainable.

1 FACTORY PATTERN (Creational)

Provides an interface for creating objects in a superclass, but allows subclasses to alter the type of objects that will be created.

Example :

ShapeFactory creates objects.

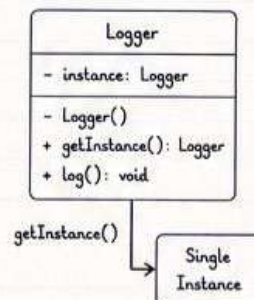


2 SINGLETON PATTERN (Creational)

Ensures a class has only one instance and provides a global point of access to it.

Example :

Logger should have only one instance.

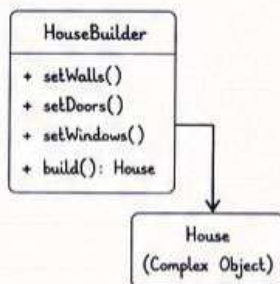


3 BUILDER PATTERN (Creational)

Separates the construction of a complex object from its representation so that the same construction process can create different representations.

Example :

Building a complex House object.

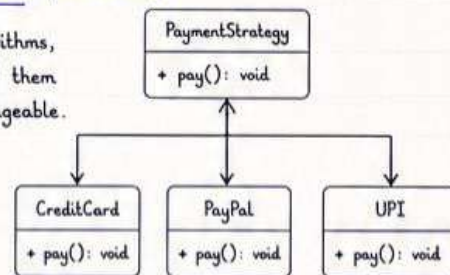


4 STRATEGY PATTERN (Behavioral)

Defines a family of algorithms, encapsulates each one of them and makes them interchangeable.

Example :

Different payment strategies.

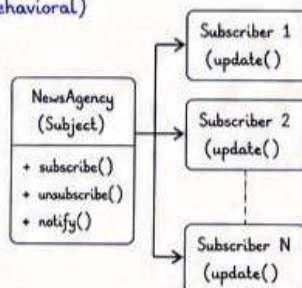


5 OBSERVER PATTERN (Behavioral)

Defines a one-to-many dependency between objects so that when one object changes state, all its dependents are notified.

Example :

NewsAgency notifies all subscribers.

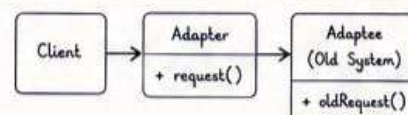


6 ADAPTER PATTERN (Structural)

Allows incompatible interfaces to work together by acting as a bridge between them.

Example :

Legacy code with new system.

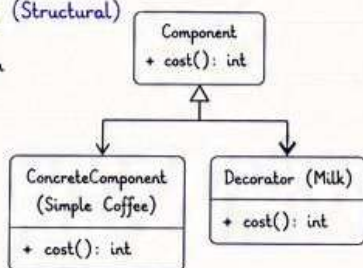


7 DECORATOR PATTERN (Structural)

Adds new functionality to an object dynamically without altering its structure.

Example :

Add toppings to a coffee.

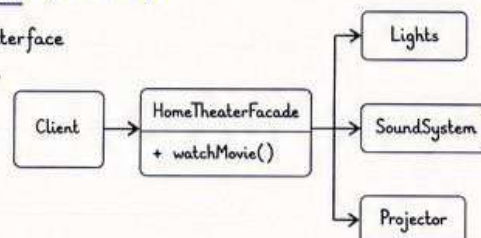


8 FACADE PATTERN (Structural)

Provides a simplified interface to a complex subsystem.

Example :

HomeTheaterFacade hides complex subsystems.



☆ QUICK COMPARISON

Pattern	Type	Purpose	Use When	Key Benefit
Factory	Creational	Create objects without specifying exact class	When object creation logic is complex	Loose coupling in object creation
Singleton	Creational	Ensure single instance of a class	When exactly one instance is needed	Global access, controlled instance
Builder	Creational	Build complex objects step by step	When object has many optional parts	Readable, flexible construction
Strategy	Behavioral	Encapsulate interchangeable algorithms	When multiple algorithms exist for a task	Easy to add new algorithms
Observer	Behavioral	Notify dependents on state change	When one-to-many dependency exists	Loose coupling, event-driven design
Adapter	Structural	Make incompatible interfaces compatible	When integrating old & new systems	Reuse existing code

HOW TO SOLVE ANY LLD INTERVIEW

"Good design is obvious.
Great design is transparent."

A STEP-BY-STEP APPROACH

Page 6 of 10



THE GOAL

Convert requirements into a clean, modular, maintainable and extensible object-oriented design.

- ✓ Meet requirements
- ✓ Handle future changes
- ✓ Write clean, testable code

1 UNDERSTAND THE REQUIREMENTS

- Listen carefully. Ask clarifying questions.
- Understand the scope and boundaries.
- Identify key functionalities and constraints.

Ask Questions Like:

- What are the main features?
- Are there any business rules?
- What are the inputs and outputs?

Think like a user and a developer.

2 IDENTIFY ENTITIES / CLASSES

- List all the nouns from the requirements.
- These will become your candidate classes.
- Remove unnecessary or duplicate entities.

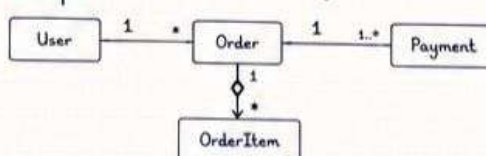
Example (Online Shopping System):



3 IDENTIFY RELATIONSHIPS

- Determine how these classes are related.
- Use association, aggregation or composition.
- Decide cardinality (1, *, 0..1, 1..* etc.).

Example:



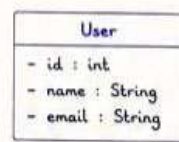
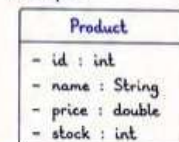
Relationship Types

- Association (uses)
- Aggregation (has-a)
- Composition (owns-a)
- Inheritance (is-a)

4 DEFINE ATTRIBUTES

- List important properties for each class.
- Consider only essential attributes.
- Avoid unnecessary getters/setters for all.

Example:



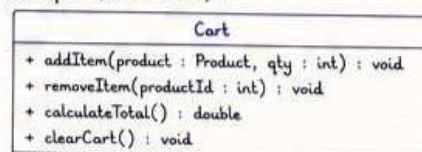
Tip

Don't add attributes just because they exist in DB.

5 DEFINE METHODS / OPERATIONS

- Define behaviors for each class.
- Follow Single Responsibility Principle.
- Methods should represent real actions.

Example (Cart Class):

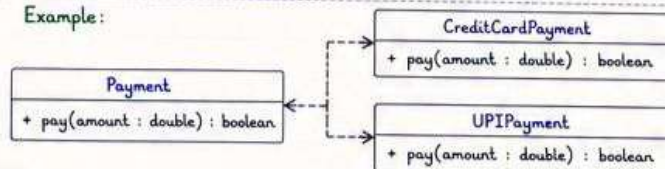


Methods are verbs.
Name methods by what they do.

6 DESIGN INTERFACES (IF NEEDED)

- Define interfaces for behaviors that may have multiple implementations.
- Helps in achieving loose coupling.

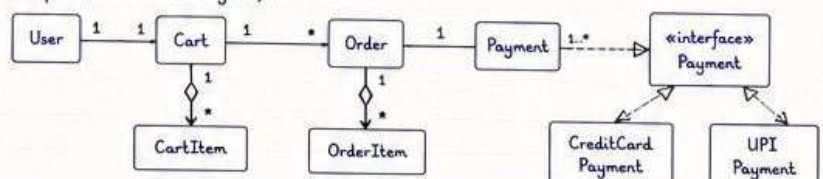
Example:



7 DRAW CLASS DIAGRAM

- Put everything together in a clean UML class diagram.
- Review relationships, multiplicity and responsibilities.

Example (Mini Class Diagram):



8 CHECK EDGE CASES

- Think of invalid inputs, failures, limits.
- Handle exceptions and validations.
- Consider concurrency if necessary.

Think About:

- Empty inputs
- Max / min limits
- Duplicate data
- Payment failures
- Network issues
- Concurrent access



Tip

Discuss edge cases in the interview.
It shows maturity.

9 REVIEW & IMPROVE

Review Checklist:

COMMON LLD PROBLEMS

— CLASSIC SYSTEMS YOU MUST PRACTICE —

"Practice common problems today, crack any design interview tomorrow."

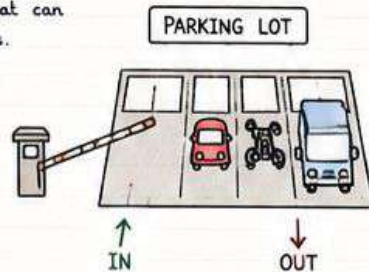
Page 7 of 10

1 PARKING LOT

Goal: Design a parking lot that can park vehicles of different sizes.

Key Entities:

- ParkingLot
- Vehicle (Car, Bike, Truck)
- ParkingSpot
- Entrance / Exit
- Ticket

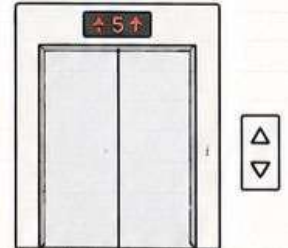


2 ELEVATOR

Goal: Design an elevator system for a building.

Key Entities:

- Elevator
- Floor
- ButtonPanel
- Direction
- Display

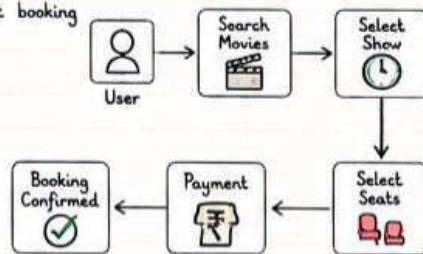


3 BOOKMYSHOW

Goal: Design a movie ticket booking system.

Key Entities:

- User
- Movie
- Show
- Seat
- Booking
- Payment

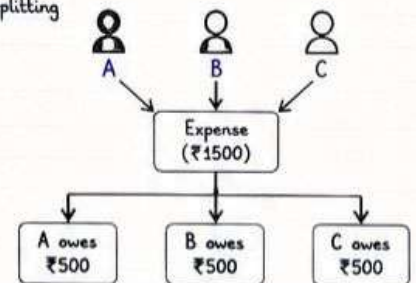


4 SPLITWISE

Goal: Design an expense splitting application.

Key Entities:

- User
- Group
- Expense
- Split
- Settlement



5 ATM

Goal: Design an ATM machine system.

Key Entities:

- ATM
- User (Bank Customer)
- Account
- Transaction
- Cash Dispenser



6 CAB BOOKING

Goal: Design a cab booking system like Uber/Ola.

Key Entities:

- User (Rider)
- Driver
- Ride
- Vehicle
- Location / Route
- Payment

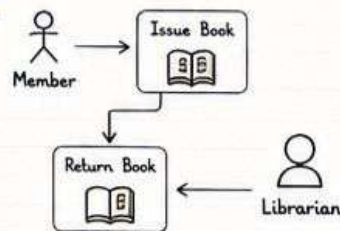


7 LIBRARY MANAGEMENT

Goal: Design a library management system.

Key Entities:

- Book
- Member
- Librarian
- Issue / Return
- Fine

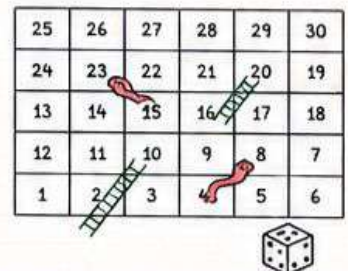


8 SNAKE & LADDER

Goal: Design the Snake and Ladder game.

Key Entities:

- Board
- Player
- Dice
- Snake
- Ladder



9 CHESS GAME

Goal: Design a chess game.

Key Entities:

- Board
- Piece (King, Queen etc.)
- Player
- Move
- Validator

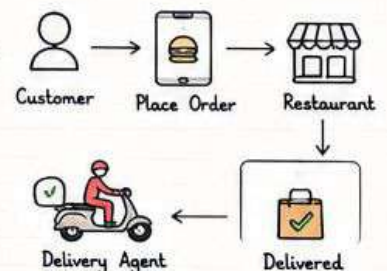


10 FOOD DELIVERY

Goal: Design a food delivery application.

Key Entities:

- Customer
- Restaurant
- Order
- Delivery Agent
- Payment



11 VENDING MACHINE

Goal: Design a vending machine system.



Why Practice These?

LLD DESIGN PATTERNS

→ PATTERNS THAT MAKE YOUR DESIGN BETTER →

"Patterns are proven solutions to recurring problems in design."

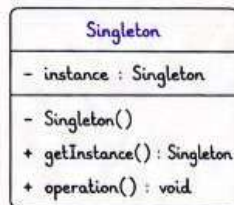
Page 8 of 10

1 SINGLETON PATTERN

Ensure a class has only one instance and provide a global point of access.

Use Cases :

- Logger
- Configuration
- Database Connection
- Cache



```

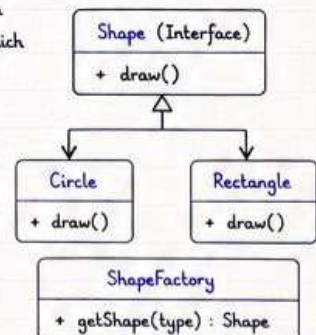
if(instance == null)
    instance = new Singleton();
return instance;
    
```

2 FACTORY PATTERN

Define an interface for creating an object, but let subclasses decide which class to instantiate.

Use Cases :

- UI Components
- Notification
- Logger
- Payment Gateways

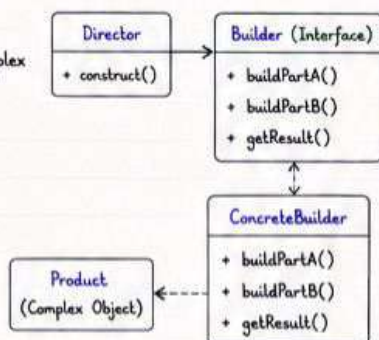


3 BUILDER PATTERN

Separate the construction of a complex object from its representation.

Use Cases :

- Complex Objects
- Report Generation
- SQL Query Builder
- Document Builder

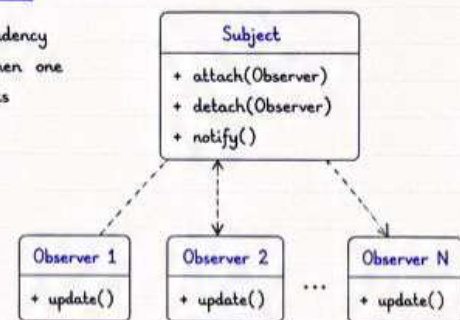


4 OBSERVER PATTERN

Define a one-to-many dependency between objects so that when one object changes state, all its dependents are notified.

Use Cases :

- Event Handling
- Notification
- Live Feeds
- Logging

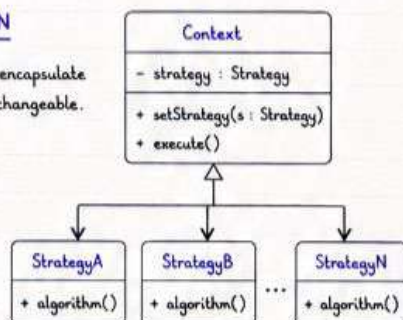


5 STRATEGY PATTERN

Define a family of algorithms, encapsulate each one, and make them interchangeable.

Use Cases :

- Payment Methods
- Sorting Algorithms
- Compression
- Validation

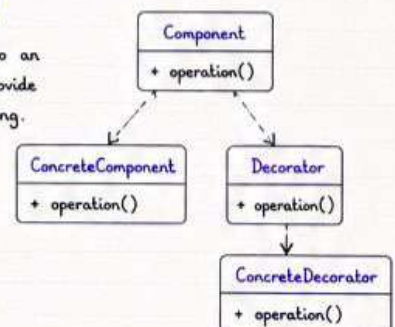


6 DECORATOR PATTERN

Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing.

Use Cases :

- Add-ons (Pizza, Coffee)
- UI Components
- Compression / Encryption
- Logging

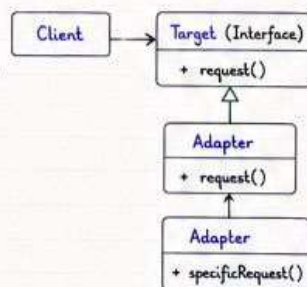


7 ADAPTER PATTERN

Convert the interface of a class into another interface clients expect.

Use Cases :

- 3rd Party Integration
- Legacy Systems
- API Integration
- Different Interfaces

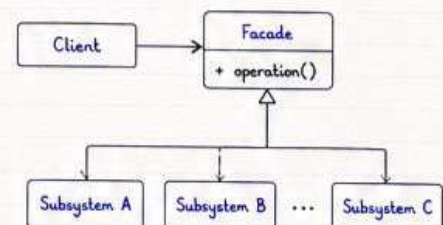


8 FACADE PATTERN

Provide a simplified interface to a complex subsystem.

Use Cases :

- Complex Subsystems
- Library APIs
- Home Theater
- Computer Startup

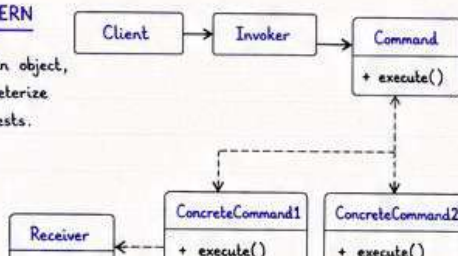


9 COMMAND PATTERN

Encapsulate a request as an object, thereby letting you parameterize clients with different requests.

Use Cases :

- Undo / Redo
- Task Scheduling
- Queue Management

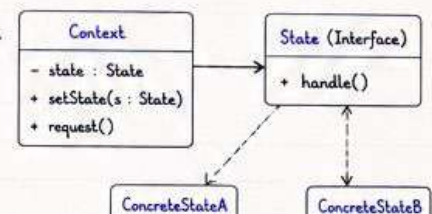


10 STATE PATTERN

Allow an object to alter its behavior when its internal state changes. The object will appear to change its class.

Use Cases :

- State Machines
- Workflow

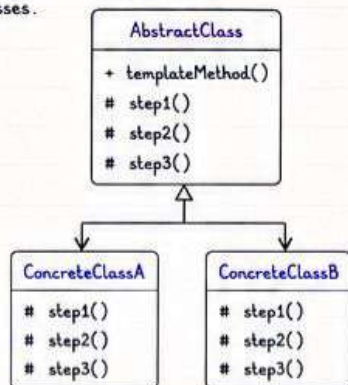


1 TEMPLATE METHOD PATTERN

Define the skeleton of an algorithm in a method, defer some steps to subclasses.

Use Cases :

- Report Generation
- Data Import / Export
- Game Development
- ETL Processes



```
// in AbstractClass
templateMethod() {
    step1();
    step2();
    step3();
}
```

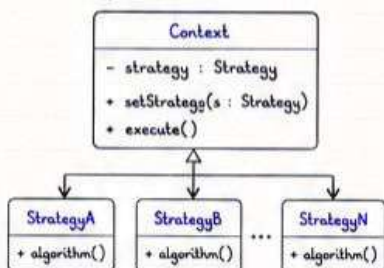
// steps implemented in subclasses

3 STRATEGY PATTERN

Define a family of algorithms, encapsulate each one, and make them interchangeable.

Use Cases :

- Payment Methods
- Sorting Algorithms
- Compression Techniques
- Shipping Calculation



```
context.setStrategy(new StrategyA());
context.execute();
```

// strategy can be changed at runtime

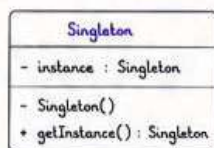
5 SINGLETON PATTERN

Ensure a class has only one instance and provide a global point of access.

Use Cases :

- Logger
- Configuration
- Cache
- Database Connection

Lazy initialization for thread unsafe (simple version)



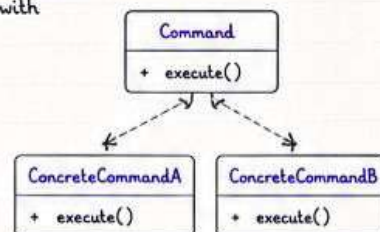
```
public static Singleton getInstance() {
    if (instance == null)
        instance = new Singleton();
    return instance;
}
```

2 COMMAND PATTERN

Encapsulate a request as an object, thereby letting you parameterize clients with different requests.

Use Cases :

- Undo / Redo
- Task Scheduling
- Remote Commands
- Menu Actions



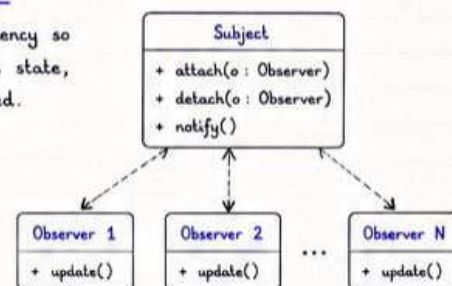
Invoker holds a command and asks it to execute. Command knows which receiver to call.

4 OBSERVER PATTERN

Define a one-to-many dependency so that when one object changes state, all its dependents are notified.

Use Cases :

- Event Handling
- Notification Systems
- Live Feeds
- Logging / Monitoring



```
notify() {
    for (Observer o : observers)
        o.update();
}
```

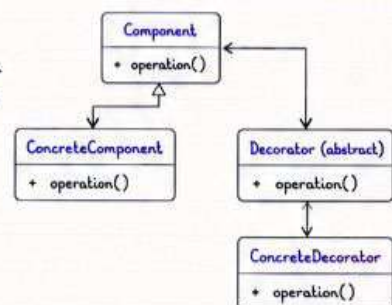
// notify all observers

6 DECORATOR PATTERN

Attach additional responsibilities to an object dynamically. Decorators provide a flexible alternative to subclassing.

Use Cases :

- Add-ons (Pizza, Coffee)
- UI Components
- Compression / Encryption
- Logging / Caching



```
operation() {
    return component.operation() + extra();
}
```

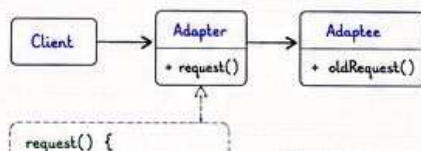
// adds extra responsibility

7 ADAPTER PATTERN

Convert the interface of a class into another interface clients expect.

Use Cases :

- 3rd Party Libraries
- Legacy Systems
- Different Interfaces
- API Integration

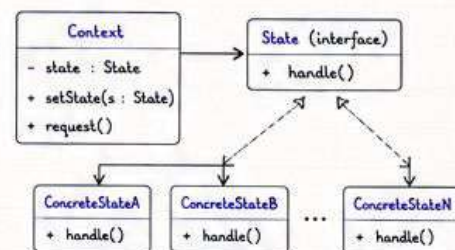


8 STATE PATTERN

Allow an object to alter its behavior when its internal state changes.

Use Cases :

- State Machines
- Workflow
- Game States
- Document States

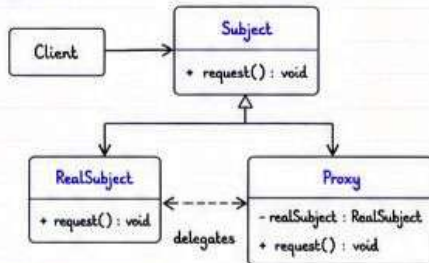


9 PROXY PATTERN

Provide a surrogate or placeholder for another object to control access to it.

Use Cases :

- Lazy Initialization
- Access Control
- Remote Proxy
- Caching



```

public void request() {
    if (realSubject == null)
        realSubject = new RealSubject();
    realSubject.request();
}

```

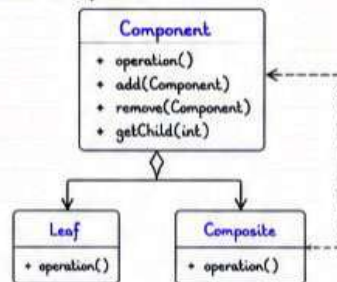
Proxy controls access to the real object.

11 COMPOSITE PATTERN

Compose objects into tree structures to represent part-whole hierarchies.

Use Cases :

- File Systems
- UI Components
- Organization Structure
- Menus and Submenus



```

public void operation() {
    for (Component child : children)
        child.operation();
}

```

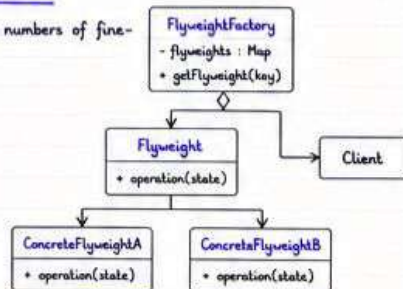
Treat leaf and composite objects uniformly.

13 FLYWEIGHT PATTERN

Use sharing to support large numbers of fine-grained objects efficiently.

Use Cases :

- Game Development
- Text Editors
- Large Scale Objects
- Memory Optimization



```

if (!flyweights.containsKey(key))
    flyweights.put(key, new ConcreteFlyweight());
return flyweights.get(key);

```

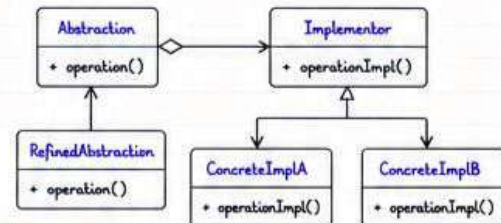
Share objects to save memory.

10 BRIDGE PATTERN

Decouple an abstraction from its implementation so that both can vary independently.

Use Cases :

- Different Platforms
- Multiple Implementations
- Decouple Abstraction from Implementation



```

abstract class Abstraction {
    protected Implementor impl;
    public void operation() {
        impl.operationImpl();
    }
}

```

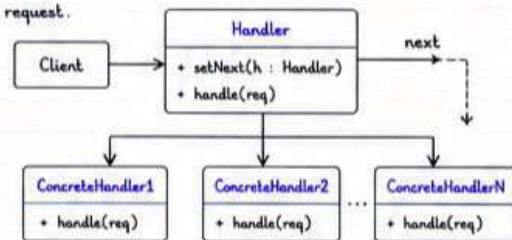
Abstraction uses Implementor to do the real work.

12 CHAIN OF RESPONSIBILITY PATTERN

Pass a request along a chain of handlers until one of them handles the request.

Use Cases :

- Logging Systems
- Exception Handling
- Request Processing
- Approval Workflows



```

public void handle(Request req) {
    if (canHandle(req))
        process(req);
    else if (next != null)
        next.handle(req);
}

```

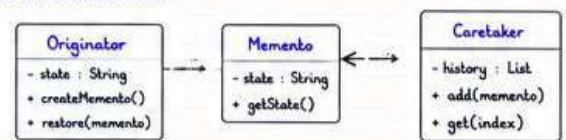
Request flows through the chain until handled.

14 MEMENTO PATTERN

Capture and externalize an object's internal state so that the object can be restored later.

Use Cases :

- Undo / Redo
- Checkpoints
- Editor History
- Game Save Points



```

Memento m = originator.createMemento();
caretaker.add(m);
originator.restore(caretaker.get(index));

```

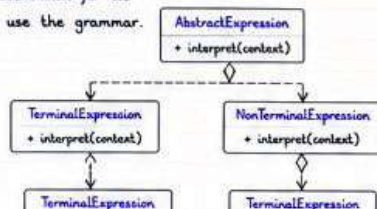
Save state externally and restore when needed.

15 INTERPRETER PATTERN

Given a language, define a representation for its grammar and an interpreter to use the grammar.

Use Cases :

- SQL Parsers
- Expression Evaluation
- Configuration Languages
- Rule Engines

**Bonus NULL OBJECT PATTERN**

Provide a default object with neutral behavior instead of null references.

Use Cases :

- Avoid Null Checks
- Optional Services
- Logging
- UI Components

